

TweetUS Airline(MongoDB)

Periodo di Riferimento: Febbraio 2015

Nazione Rilevazione Tweet: USA

Totale Tweet: 14.848

Connessione al DataSet

```
import pymongo  
from pymongo import MongoClient  
  
connection =  
MongoClient("mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=  
2000")  
  
db = connection.Tweet
```

Il dataset in formato csv è stato importato su MongoDB Compass.

Nome del dataset in MongoDB Compass: Tweet

Nome della collezione: tweet

Rappresentazione TweetUS Airline in MongoDB(NOSQL)

Query

```
database = db.tweet.find().limit(1)  
  
for output in database:  
    print(output)
```

Output Query

```
{ '_id': ObjectId('6054c8872e7a8b10444e4254'), 'tweet_id': '570306133677760513', 'airline_sentiment': 'neutral', 'airline_sentiment_confidence': '1.0', 'airline': 'Virgin America', 'name': 'cairdin', 'retweet_count': '0', 'text': '@VirginAmerica What @dhepburn said.', 'tweet_created': '2015-02-24 11:35:52 -0800', 'user_timezone': 'Eastern Time (US & Canada)'} }
```

Spiegazione DataSet

Nel caso di database NOSQL, le informazioni non trovano più posto in righe elencate in tabelle, ma in oggetti completamente diversi e non necessariamente strutturati, come ad esempio documenti archiviati in collezioni. Nei dbms NOSQL a documenti, è importante notare l'assenza delle relazioni. Uno dei due meccanismi con cui vengono collegate le informazioni è il Referencing. Il referencing, consiste nel fare in modo che un documento contenga, tra i suoi dati, l'id di un altro documento. Questo meccanismo è molto utile per realizzare relazioni molti-a-molti.

Le variabili più 'importanti' del dataset e che utilizzeremo nelle query sono:

1)airline_sentiment(Positivo,Negativo e Neutrale): Ogni tweet è classificato nelle tre categorie precedentemente citate

2)airline: Rappresenta il nome della compagnia aerea. Ogni tweet è riferito ad una determinata compagnia aerea

3)name: Nome Utente

4)retweet_count: Il numero di volte in cui un tweet specifico è stato ritwittato.

5)text: Commento dell'utente

6)negativereason: Ragione Negativa(Esempio: Bad Flight)

7)user_timezone: Indica il fuso orario dell'utente

Analitica 1

Quanti tweet per ciascun utente ?

La variabile da prendere in considerazione è name, che identifica il nome di un utente.

Query 1

```
query1 = db.tweet.aggregate([{"$group": {"_id": "$name", "total": {"$sum": 1}}, {"$sort": {"total": -1}}, {"$limit": 10 } ])
```

```
for output in query1:
```

```
    print(output)
```

Output Query 1

```
{ '_id': 'JetBlueNews', 'total': 63 }
{ '_id': 'kbosspotter', 'total': 32 }
{ '_id': '_mhertz', 'total': 29 }
{ '_id': 'otisday', 'total': 28 }
{ '_id': 'throthra', 'total': 27 }
{ '_id': 'weezerandburnie', 'total': 23 }
{ '_id': 'rossj987', 'total': 23 }
{ '_id': 'GREATNESSEOA', 'total': 22 }
{ '_id': 'MeeestarCoke', 'total': 22 }
{ '_id': 'scoobydoo9749', 'total': 21 }
```

Per comodità di visualizzazione ho modificato la query in modo tale da visualizzare in ordine decrescente(per numero di tweet scritti) i primi dieci utenti.

Commento Query 1

La Aggregation Pipeline ci consente di trasformare e combinare i documenti di una collezione. Utilizziamo l'operatore \$group che è del tutto analogo al GROUP BY di SQL. Lo usiamo per creare un nuovo documento con un campo _id che indica quale è il campo in base al quale raggruppiamo. In questo caso raggruppiamo per name(nome utente).Aggiungiamo 1 tramite (\$sum: 1) per ogni documento in cui name(nome utente) è uguale al nome utente per cui stiamo raggruppando.

Query 1 Originale

```
query1 = db.tweet.aggregate([{"$group": {"_id": "$name", "total": {"$sum": 1}}]])
for output in query1:
    print(output)
```

Analitica 2

Quanti retweet per ciascuna compagnia aerea?

Le variabili da prendere in considerazione sono :

1) retweet_count

2) airline

retweet_count ci dice quante volte un determinato tweet è stato 'ritwittato', mentre airline indica il nome della compagnia aerea.

Query 2

```
query2 = db.tweet.aggregate([ {'$group': { '_id': '$airline', 'total': { '$sum': { '$convert': { 'input': '$retweet_count', 'to': 'int' }}}}}, {'$sort': {'total': -1}}])
```

for output in query2:

```
    print(output)
```

Output Query 2

```
{ '_id': 'United', 'total': 421 }
{ '_id': 'Delta', 'total': 252 }
{ '_id': 'US Airways', 'total': 249 }
{ '_id': 'Southwest', 'total': 145 }
{ '_id': 'American', 'total': 117 }
{ '_id': 'Virgin America', 'total': 26 }
```

Per comodità di visualizzazione ho modificato la query in modo tale da visualizzare in ordine decrescente(per numero di retweet) le compagnie aeree.

Commento Query 2

La logica da seguire in questo caso è simile alla logica della query precedente. Raggruppiamo per airline(nome compagnia aerea) e sommiamo il numero di retweet riferiti alla compagnia aerea per cui stiamo raggruppando.

Dato che il campo retweet_count non è di tipo numerico, è necessario adoperare una conversione. Convertiamo l'input, in questo caso retweet_count in un intero in modo tale da applicare senza problemi l'operatore \$sum per effettuare la somma.

Analitica 3

TagCloud dei Termini

La variabile da utilizzare è text

Query 3

```
import numpy as np
import pandas as pd
from os import path
from PIL import Image
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator

import matplotlib.pyplot as plt

query3 = db.tweet.find({}, {"text":1, "_id":0})
comments = ""
stopwords = set(STOPWORDS)

query3 = list(query3)
for doc in query3:
    comment=doc['text']
    comments+=comment+""

worldcloud= WordCloud (width=800, height=200,background_color='white', stopwords=stopwords,
min_font_size=10).generate(comments)

plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(worldcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

Output Query 3

for doc in query3:

```
    comment=doc['text'] comme
```

```
    comments+=comment+""
```

In output abbiamo:

```
@VirginAmerica What @dhepburn said.@VirginAmerica plus you've added commercials to the experience... tacky.
```

Con i comandi successivi andiamo a creare la TagCloud dei termini.

Analitica 4

Quale è la compagnia con il maggior numero di tweet con sentiment negativa?

Le variabili da prendere in considerazione sono:

1)airline

2) airline_sentiment

airline identifica il nome della compagnia aerea, mentre airline_sentiment identifica il sentiment degli utenti di twitter nei confronti di una compagnia aerea.

Query 4

```
query4 = db.tweet.aggregate([ { "$match": { "airline_sentiment": "negative" } }, { "$group": { "_id": "$airline", "total": { "$sum": 1 } } }, { "$sort": {"total":-1}}, { "$limit":1}])
```

```
for output in query4:
```

```
    print(output)
```

Output Query 4

```
{ '_id': 'United', 'total': 2633 }
```

Commento Query 4

Un altro operatore di pipeline che possiamo usare oltre \$group è \$match, che equivale al metodo find e ci consente di aggregare solo i documenti che soddisfano certi criteri o, al contrario, di escludere documenti dal risultato. In questo caso aggregiamo solo quei documenti il cui campo airline_sentiment è uguale a negative. Successivamente la logica da adoperare è la stessa delle query precedenti. Raggruppiamo per airline(nome della compagnia) e aggiungiamo 1 tramite (\$sum: 1) per ogni documento in cui airline(nome della compagnia) è uguale al nome della compagnia per cui stiamo per cui stiamo raggruppando. \$sort e \$limit combinati consentono di rispondere alle classiche domande del tipo “trova i primi N elementi”, in questo caso ordiniamo il totale dei tweet negativi per un data compagnia aerea in senso decrescente e con limit restituiamo la compagnia aerea che presenta il maggior numero di tweet negativi.

Analitica 5

Distribuzione sentiment per ciascuna compagnia aerea

Le variabili da prendere in considerazione sono:

- 1) airline
- 2) airline_sentiment

airline identifica il nome della compagnia aerea mentre airline_sentiment identifica il sentiment degli utenti di twitter nei confronti di una compagnia aerea.

Query 5

```
query5 = db.tweet.aggregate([{"$group":{"_id": "$airline", "CountPositive": {"$sum": {"$cond": [{"$eq": ["$airline_sentiment", "positive"]}, 1, 0 ]}}, "CountNegative": {"$sum": {"$cond": [{"$eq": ["$airline_sentiment", "negative"]}, 1, 0 ]}}, "CountNeutral": {"$sum": {"$cond": [{"$eq": ["$airline_sentiment", "neutral"]}, 1, 0 ]}} } ]])
```

for output in query5:

```
print(output)
```

Output Query 5

```
{'_id': 'Southwest', 'CountPositive': 570, 'CountNegative': 1186, 'CountNeutral': 664}
{'_id': 'American', 'CountPositive': 336, 'CountNegative': 1960, 'CountNeutral': 463}
{'_id': 'Delta', 'CountPositive': 544, 'CountNegative': 955, 'CountNeutral': 723}
{'_id': 'United', 'CountPositive': 492, 'CountNegative': 2633, 'CountNeutral': 697}
{'_id': 'Virgin America', 'CountPositive': 152, 'CountNegative': 181, 'CountNeutral': 171}
{'_id': 'US Airways', 'CountPositive': 269, 'CountNegative': 2263, 'CountNeutral': 381}
```


Commento Query 5

La logica da seguire in questo caso è simile alle query precedenti. Raggruppiamo per airline(nome della compagnia aerea) e definiamo CountPositive, CountNegative e CountNeutral. In questo caso andremo a sommare(\$sum) solo quando una condizione risulta essere vera, nel caso di CountPositive la condizione(\$cond) specifica da utilizzare è (\$eq). La condizione risulta essere vera quando il campo airline_sentiment è uguale a positive .Solo in questo caso il campo CountPositive viene incrementato di 1. La stessa logica viene utilizzata per CountNegative e CountNeutral. Così facendo, otterremo la distribuzione del sentiment, per ogni compagnia aerea.

Analitica 6

Distribuzione del sentiment per ciascuna motivazione

La variabile da prendere in considerazione è negativereason

Query 6

```
query6 = db.tweet.aggregate([{"$match":{"airline_sentiment":"negative"}}, {"$group": {"_id" : "$negativereason" , "total" : {"$sum":1}}}, {"$sort":{"total":-1}} ])
```

for output in query6:

```
print(output)
```

Output Query 6

```
{'_id': 'Customer Service Issue', 'total': 2910}
{'_id': 'Late Flight', 'total': 1665}
{'_id': 'Can't Tell', 'total': 1190}
{'_id': 'Cancelled Flight', 'total': 847}
{'_id': 'Lost Luggage', 'total': 724}
{'_id': 'Bad Flight', 'total': 580}
{'_id': 'Flight Booking Problems', 'total': 529}
{'_id': 'Flight Attendant Complaints', 'total': 481}
{'_id': 'longlines', 'total': 178}
{'_id': 'Damaged Luggage', 'total': 74}
```

Commento Query 6

In questo caso aggregiamo solo quei documenti il cui campo `airline_sentiment` è uguale a `negative`. Successivamente la logica da adoperare è la stessa delle query precedenti. Raggruppiamo per `negativereason` (ragioni negative) e aggiungiamo 1 tramite (`$sum: 1`) per ogni documento in cui `negativereason` (ragioni negative) è uguale alla `negativereason` per cui stiamo per cui stiamo raggruppando.

Analitica 7

Autori, in ordine alfabetico, che hanno ricevuto più di 10 retweet

Le variabili da prendere in considerazione sono :

1) `name`

2) `retweet_count`

`name` identifica l'autore del tweet, mentre `retweet_count` ci dice quante volte un determinato tweet è stato 'ritwittato'.

Query 7

```
query7 = db.tweet.aggregate([{'$group': { '_id': '$name', 'total': {'$sum': {'$convert': { 'input': '$retweet_count', 'to': 'int' } } } }}, {'$match': {'total': {'$gt': 10}}}, {'$sort': {'_id': 1}}])
```

```
for output in query7:
```

```
    print(output)
```

Output Query 7

```
{ '_id': 'AbeFroman', 'total': 18 }
{ '_id': 'JasonShaw2', 'total': 31 }
{ '_id': 'NickTypesWords', 'total': 20 }
{ '_id': 'No_Cut_Card', 'total': 15 }
{ '_id': 'OBJ_3', 'total': 80 }
{ '_id': 'OlivierGachot', 'total': 11 }
{ '_id': 'TatianaKing', 'total': 31 }
{ '_id': 'TheBuie', 'total': 22 }
{ '_id': 'airliners2', 'total': 28 }
{ '_id': 'davidgoodson71', 'total': 13 }
{ '_id': 'seattlefordguy', 'total': 22 }
```

Commento Query 7

La logica da adoperare è la stessa delle query precedenti. Raggruppiamo per name(nome autore) e convertiamo l'input, in questo caso retweet_count in un intero in modo tale da applicare senza problemi l'operatore \$sum per effettuare la somma. Successivamente(considerando il nome utente per il quale stiamo raggruppando) andiamo ad aggregare solo quei documenti che hanno ricevuto più di 10 retweet(\$gt: **greater than**).

Analitica 8

Data in input una compagnia aerea, mostrare la TagCloud dei termini utilizzati in caso di sentiment negativa.

Query 8

```
import numpy as np
import pandas as pd
from os import path
from PIL import Image
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator

import matplotlib.pyplot as plt

nome_compagnia = input("Inserisci il nome di una compagnia aerea:\t")

query8 = db.tweet.find({"$and": [{ "airline_sentiment": "negative"}, {"airline": nome_compagnia}]},
{"text":1,"_id":0,"airline":1})

comments = ""
stopwords = set(STOPWORDS)

query8 = list(query8)
for doc in query8:
    comment=doc['text']
    comments+=comment+""

worldcloud= WordCloud (width=800, height=200,background_color='white', stopwords=stopwords,
min_font_size=10).generate(comments)

plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(worldcloud)
```

```
plt.axis("off")  
plt.tight_layout(pad=0)  
plt.show()
```

Inserimento nome compagnia in input

Inserisci il nome di una compagnia aerea:

Delta

Output Query 8



Analitica 9

Data in input una compagnia aerea, mostrare la TagCloud dei termini utilizzati in caso di sentiment positiva.

Query 9

```
nome_compagnia = input("Inserisci il nome di una compagnia aerea:\t")
```

```
query9 = db.tweet.find({"$and": [{ "airline_sentiment": "positive"}, {"airline": nome_compagnia}]}, {"text": 1, "_id": 0, "airline": 1})
```

```
comments = ""
```

```
stopwords = set(STOPWORDS)
```

```
query9 = list(query9)
```

```
comment=doc['text']  
comments+=comment+""
```

```
plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(worldcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

Inserisci il nome di una compagnia aerea:

[illegible]

Totale Tweet: 14.848

In questo secondo progetto ho seguito la logica della "Normalizzazione" in quanto nel dataset Tweet.csv si hanno informazioni relative a tre entità differenti: tweet, autore, airline. Il dataset denominato tweet, presenta al suo interno tutte le variabili riferite all'entità 'tweet', mentre il dataset denominato users, presenta al suo interno tutte le variabili riferite all'entità utente. Infine abbiamo il dataset denominato airlines, in cui tutte le variabili fanno riferimento all'entità airline.

Load tweet.csv

```
load csv with headers from 'file:///tweet.csv' as line create (t:Tweet {ID_Tweet:line.tweet_id, Airline_Sentiment:line.airline_sentiment, Airline_Sentiment_Conf:toFloat(line.airline_sentiment_confidence), Negative_Reason:line.negative_reason, Negative_Reason_Conf:toFloat(line.negative_reason_confidence), Airline_Sentiment_Gold:line.airline_sentiment_gold, Negative_Reason_Gold:line.negative_reason_gold, Retweet_Count:toInteger(line.retweet_count), Text:line.text, Coord_Tweet:line.tweet_coord, Tweet_Created:line.tweet_created, Tweet_Loc:line.tweet_location, User_Timezone:line.user_timezone, id_compagnia:line.ID_Compagnia, id_utente:line.ID_UTENTE})
```

Nel dataset tweet.csv sono state aggiunte due colonne:

1)id_compagnia

2)id_utente

id_compagnia identifica in modo univoco la compagnia aerea, mentre id_utente identifica in modo univoco l'utente che ha scritto i tweet. Questo meccanismo ci servirà per collegare correttamente i varie nodi tra di loro attraverso le relazioni che andremo a creare successivamente. In questo caso ogni riga rappresenta un nodo.

Load users.csv

```
load csv with headers from 'file:///users.csv' as line create (u:Users {nome_utente:line.Nome_Utente, id_utente:line.ID_UTENTE})
```

Load airlines.csv

```
load csv with headers from 'file:///airlines.csv' as line create (a:Airlines {nome_compagnia:line.Nome_Compagnia, id_compagnia:line.ID_Compagnia})
```

Creazione Relazioni

```
MATCH (t:Tweet)
MATCH (u:Users)
WHERE t.id_utente = u.id_utente
CREATE (u)-[r: HASCITTO]->(t)
```

In questo caso abbiamo creato la relazione 'HASCritto'. Un utente ha scritto un tweet. Questo utente è identificato da un id_utente. Se l'id_utente presente nel dataset tweet è uguale a uno dei id_utenti presenti nel dataset users, allora l'utente in questione esiste. Questo utente ha scritto un tweet.

```
MATCH (t:Tweet)
where t.Retweet_Count > 0
create (t)-[r2:RETWEETED]->(t)
```

In questo caso abbiamo creato la relazione 'RETWEETED'. Se il valore di Retweet_Count è maggiore di 0 allora vorrà dire che quel tweet identificato da uno specifico id, è stato ritwittato.

```
Match (t:Tweet)
Match(a:Airlines)
Where t.id_compagnia = a.id_compagnia
CREATE(t)-[r3:RIFERITO]->(a)
```

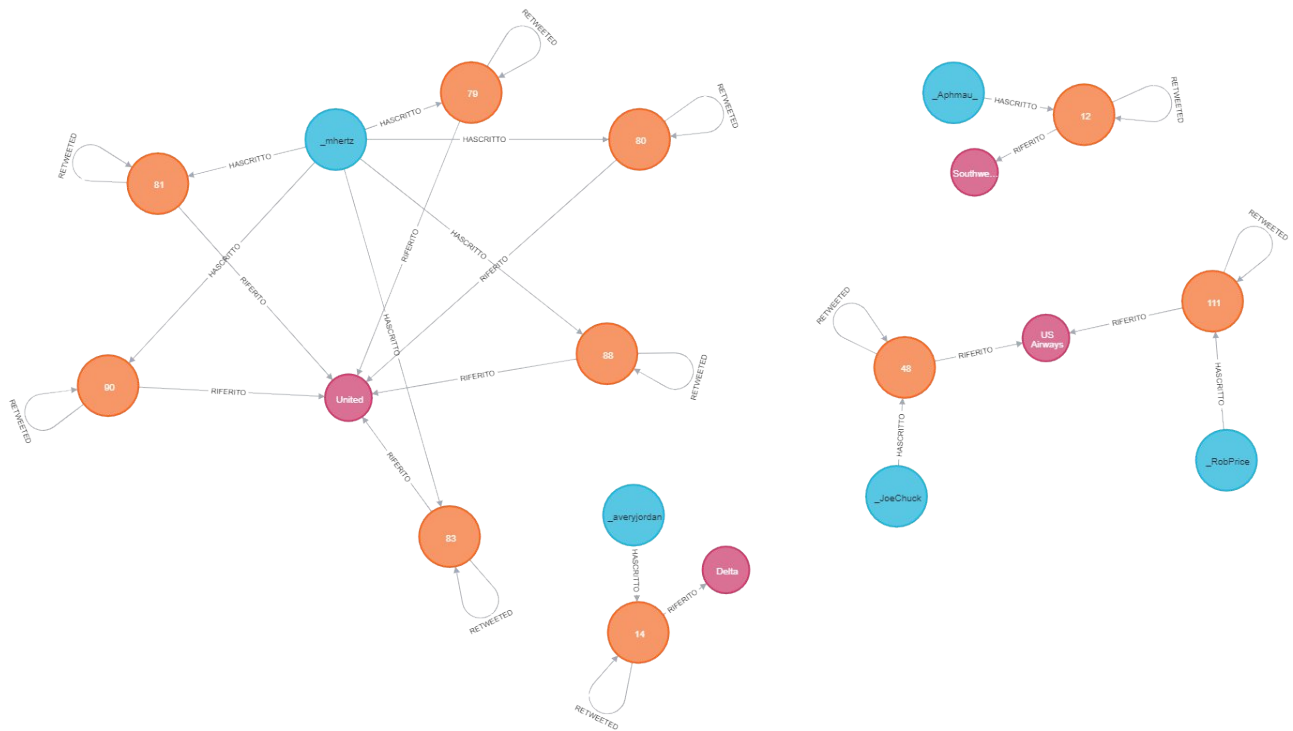
In questo caso abbiamo creato la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Se l'id_compagnia presente nel dataset tweet è uguale a uno dei id_compagnia presenti nel dataset airlines, allora la compagnia aerea in questione esiste. Questo tweet è riferito ad una compagnia aerea.

Rappresentazione TweetUS Airline(NEO4J)

Query

```
match (u:Users)-[r:HASCritto]->(t:Tweet)-[r1:RETWEETED]-(t:Tweet)-[r2:RIFERITO]->(a:Airlines) return u,r,t,r1,r2,a limit 10
```

Output Query



Nodo Arancione: <id> del tweet

Nodo Azzurro: Nome del utente che ha scritto il tweet

Nodo Viola: Nome compagnia aerea

Relazioni: HASCRITTO,RETWEETED,RIFERITO.

Analitica 1

Quanti tweet per ciascun utente ?

Query 1

```
match (u:Users)-
[:HASCRITTO]->(t:Tweet) with u.nome_utente as Nome_Utente,co
unt(*) as Totale_Tweet return Nome_Utente,Totale_Tweet order by Totale_Tweet des
c limit 10
```


Output Query 1

	Nome_Utente	Totale_Tweet
1	"JetBlueNews"	63
2	"kbosspotter"	32
3	"_mhertz"	29
4	"otisday"	28
5	"throthra"	27
6	"weezerandburnie"	23
7	"rossj987"	23
8	"MeeestarCoke"	22
9	"GREATNESSEOA"	22
10	"scoobydoo9749"	21

Commento Query 1

Sfruttiamo la relazione 'HASCritto'. Un utente HASCritto un tweet. Raggruppiamo per nome utente mediante la clausola with e rinominiamo tramite la clausola as (che rappresenta l'alias), la colonna che in output conterrà il nome degli Utenti.(Nome_Utente). count(*) rappresenta una funzione di aggregazione che conterà il numero di volte in cui un Nome_Utente specifico compare. Così facendo ,otterremo il numero di tweet per ogni utente.

Analitica 2

Quanti retweet per ciascuna compagnia aerea?

Query 2

```
match (t:Tweet)-[:RETWEETED]->(t:Tweet)-  
[:RIFERITO]->(a:Airlines) return a.nome_compagnia as NomeCompagn  
ia, sum(t.Retweet_Count) as TotaleRetweet order by TotaleRetweet desc
```

Output Query 2

	NomeCompagnia	TotaleRetweet
1	"United"	421
2	"Delta"	252
3	"US Airways"	249
4	"Southwest"	145
5	"American"	117
6	"Virgin America"	26

Commento Query 2

Sfruttiamo la relazione 'RETWEETED'. Se il valore di Retweet_Count è maggiore di 0 allora vorrà dire che quel tweet identificato da uno specifico id, è stato ritwittato.

Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con return a.nome_compagnia raggruppiamo per nome_compagnia e utilizziamo la clausola as (che rappresenta l'alias), per rinominare la colonna associata a nome_compagnia. Con sum(t.Retweet_Count), sommiamo il numero di retweet che si riferiscono alla compagnia aerea per cui stiamo raggruppando. Con order by TotaleRetweet desc ordiniamo i retweet totali in ordine decrescente.

Query 3 – Query 8 – Query 9 (Argomento: TagCloud dei Termini)

Mancanti per difficoltà nel creare la TagCloud. Alla fine del file word, ci sono tre query non assegnate nel progetto, che ho creato per sopperire alla mancanza di queste tre query non svolte.

Analitica 4

Quale è la compagnia con il maggior numero di tweet con sentiment negativa?

Query 4

```
match (t:Tweet)-  
[:RIFERITO]->(a:Airlines) where t.Airline_Sentiment = "negati  
ve" return a.nome_compagnia, count(t) order by count(t) desc limit(1)
```

Output Query 4

	a.nome_compagnia	count(t)
1	"United"	2633

Commento Query 4

Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con la clausola where troviamo tutti quei nodi di tipo tweet il cui campo Airline_Sentiment è uguale a negative. Con return a.nome_compagnia, raggruppiamo per compagnia aerea, e con count(t) contiamo tutti quei nodi (che si riferiscono alla compagnia aerea per cui stiamo raggruppando), che presentano il campo Airline_Sentiment uguale a negative. Con order by count(t) desc ordiniamo le compagnie aeree in base al loro numero di tweet negativi ricevuti in ordine decrescente. Con limit(1), prendiamo la compagnia aerea che presenta il numero di tweet negativi più alto.

Analitica 5

Distribuzione sentiment per ciascuna compagnia aerea

Query 5

```
match (t:Tweet)-  
[:RIFERITO]->(a:Airlines) return t.Airline_Sentiment as Sentiment,a.n  
ome_compagnia as NomeCompagnia, count(t) as Conteggio
```

Output Query 5

Table

A

Text

>_

Code

"Sentiment"	"NomeCompagnia"	"Conteggio"
"negative"	"Virgin America"	181
"neutral"	"Virgin America"	171
"positive"	"Virgin America"	152
"negative"	"United"	2633
"positive"	"United"	492
"neutral"	"United"	697
"positive"	"Southwest"	570
"negative"	"Southwest"	1186
"neutral"	"Southwest"	664
"negative"	"Delta"	955
"neutral"	"Delta"	723

"positive"	"Delta"	544
"neutral"	"US Airways"	381
"negative"	"US Airways"	2263
"positive"	"US Airways"	269
"negative"	"American"	1960
"neutral"	"American"	463
"positive"	"American"	336

Commento Query 5

Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con return t.Airline_Sentiment ,a.nome_compagnia, raggruppiamo per sentiment(Positivo,Negativo,Neutrale) e per compagnia. Con count(t) contiamo il numero di tweet relativi alla compagnia aerea e al sentiment per cui stiamo raggruppando.

Analitica 6

Distribuzione del sentiment per ciascuna motivazione

Query 6

```
match (u:Users)-
[:HASCRITTO]->(t:Tweet) where t.Airline_Sentiment = "negative" retu
rn t.Negative_Reason, count(t) as Totale order by Totale desc
```

Output Query 6

Table

Text

Code

"t.Negative_Reason"	"Totale"
"Customer Service Issue"	2910
"Late Flight"	1665
"Can't Tell"	1190
"Cancelled Flight"	847
"Lost Luggage"	724
"Bad Flight"	580
"Flight Booking Problems"	529
"Flight Attendant Complaints"	481
"longlines"	178
"Damaged Luggage"	74

Commento Query 6

Sfruttiamo la relazione 'HASCritto'. Un utente HASCritto un tweet. Con la clausola where troviamo tutti quei nodi di tipo tweet il cui campo Airline_Sentiment è uguale a negative. Con return t.Negative_Reason, raggruppiamo per Negative_Reason(Ragione Negativa), e con count(t) contiamo tutti quei nodi (che si riferiscono alla ragione negativa per cui stiamo raggruppando),che presentano il campo Airline_Sentiment uguale a negative. Con order by Totale desc, ordiniamo il Totale delle ragioni negative in modo decrescente.

Analitica 7

Autori, in ordine alfabetico, che hanno ricevuto più di 10 retweet

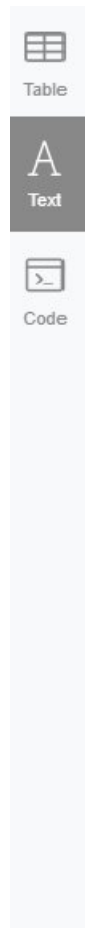
Query 7

```
match (t:Tweet)-[:RETWEETED]->(t:Tweet)<-[:HASCritto]-  
(u:Users) with u.nome_utente as NomeUtente, sum(t.Retweet_Count) as TotaleRetweet where  
TotaleRetweet > 10 return NomeUtente, TotaleRetweet order by NomeUtente
```

Output Query 7

Commento Query 7

Sfruttiamo la relazione 'RETWEETED'. Se il valore di RetweeT_Count è maggiore di 0 vorrà dire che quel tweet identificato da uno specifico id, è ritwittato. Sfruttiamo la relazione 'HASCritto'. Un utente HASCritto tweet. Raggruppiamo per nome per ogni utente per cui stiamo raggruppando, sommiamo il numero retweet con il comando con sum(t.RetweeT_Count). Con la where troviamo tutti quei nodi di tweet il cui campo t.RetweeT_Count maggiore di dieci. Con order by NomeUtente, ordiniamo i nomi degli In questo caso verranno stampati in prima i nomi degli utenti che con una lettera Maiuscola.



"NomeUtente"	"TotaleRetweet"
"AbeFroman"	18
"JasonShaw2"	31
"NickTypesWords"	20
"No_Cut_Card"	15
"OBJ_3"	80
"OlivierGachot"	11
"TatianaKing"	31
"TheBuie"	22
"airliners2"	28
"davidgoodson71"	13
"seattlefordguy"	22

allora
stato
un
utente e
di
clausola
tipo
è
Utenti.
output
iniziano

Analitiche nuove

Analitica 1

Contare il numero totale di tweet per compagnia aerea e per fuso orario del utente(User_Timezone).

Query 1

```
match (u:Users)-[:HASCritto]->(t:Tweet)-  
[:RIFERITO]->(a:Airlines) where t.User_Timezone <> "" and t.Airline_Se  
ntiment = "negative" with t.User_Timezone as ZonaUsa, a.nome_compagnia as NomeC  
ompagnia ,count(*) as TotaleTweet return ZonaUsa, NomeCompagnia, TotaleTweet orde  
r by TotaleTweet desc limit (10)
```

Output Query 1

Table		
Text		
Code		
	"ZonaUsa"	"NomeCompagnia" "TotaleTweet"
	"Eastern Time (US & Canada)"	"US Airways" 682
	"Eastern Time (US & Canada)"	"United" 594
	"Eastern Time (US & Canada)"	"American" 383
	"Eastern Time (US & Canada)"	"Delta" 383
	"Central Time (US & Canada)"	"United" 334
	"Central Time (US & Canada)"	"American" 321
	"Pacific Time (US & Canada)"	"United" 285
	"Central Time (US & Canada)"	"Southwest" 249
	"Eastern Time (US & Canada)"	"Southwest" 223
	"Central Time (US & Canada)"	"US Airways" 213

Commento Query 1

Sfruttiamo la relazione 'HASCritto'. Un utente HASCritto un tweet. Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con la clausola where troviamo tutti quei nodi in cui il campo User_Timezone non sia vuoto e il cui campo Airline_Sentiment è uguale a negative. Successivamente raggruppiamo per User_Timezone e per compagnia aerea e contiamo il numero totale di tweet. Ordiniamo il numero totale di tweet in ordine decrescente e con limit 10, visualizziamo i primi 10 nodi del elenco.

Dalla tabella si capisce che gli utenti più attivi(dai dati che abbiamo a disposizione) si trovano nel EST degli Stati Uniti.

Analitica 2

Considerando il fuso orario del Eastern Time (Us & Canada), raggruppare per Negative_Reason e contare il totale della Negative_Reason per la quale si sta raggruppando.

Query 2

```
match (u:Users)-[:HASCritto]->(t:Tweet)-[:RIFERITO]->(a:Airlines) where t.User_Timezone = "Eastern Time (US & Canada)" and t.Negative_Reason <> "" with t.Negative_Reason as RagioneNegativa, count(*) as Totale return RagioneNegativa, Totale order by Totale desc
```

Output Query 2

RagioneNegativa	Totale
Customer Service Issue	725
Late Flight	442
Can't Tell	290
Cancelled Flight	224
Lost Luggage	200
Bad Flight	138
Flight Attendant Complaints	120
Flight Booking Problems	113
longlines	41
Damaged Luggage	20

Commento Query 2

Sfruttiamo la relazione 'HASCritto'. Un utente HASCritto un tweet. Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con la clausola where troviamo tutti quei nodi in cui il campo User_Timezone è uguale a Eastern Time (US & Canada) e il campo Negative_Reason diverso da "" (Spazio Vuoto). Successivamente raggruppiamo per Negative_Reason e contiamo il numero totale della Negative_Reason per cui stiamo raggruppando. Ordiniamo il numero totale di Negative_Reason in ordine decrescente.

Dalla tabella si capisce che la Negative_Reason più utilizzata è: Customer Service Issue (Problema con il servizio clienti).

Analitica 3

Considerando la compagnia aerea United, contare il totale della Negative_Reason per la quale si sta raggruppando.

Query 3

```
match (u:Users)-[:HASCRITTO]->(t:Tweet)-[:RIFERITO]->(a:Airlines) where t.Negative_Reason <> "" and a.name_compagnia = "United" with t.Negative_Reason as RagioneNegativa, count(*) as Totale return RagioneNegativa, Totale order by Totale desc
```

Output Query 3

Table	
Text	
Code	
"RagioneNegativa"	"Totale"
"Customer Service Issue"	681
"Late Flight"	525
"Can't Tell"	379
"Lost Luggage"	269
"Bad Flight"	216
"Cancelled Flight"	181
"Flight Attendant Complaints"	168
"Flight Booking Problems"	144
"longlines"	48
"Damaged Luggage"	22

Commento Query 3

Sfruttiamo la relazione 'HASCRITTO'. Un utente HASCRITTO un tweet. Sfruttiamo la relazione 'RIFERITO'. Il tweet è riferito ad una compagnia aerea. Con la clausola

where troviamo tutti quei nodi in cui il campo nome_compagnia è uguale a United, e il campo Negative_Reason diverso da ""(Spazio Vuoto).Successivamente raggruppiamo per Negative_Reason e contiamo il numero totale della Negative_Reason per cui stiamo raggruppando. Ordiniamo il numero totale di Negative_Reason in ordine decrescente.

Dalla tabella si capisce che la Negative_Reason più utilizzata è: Customer Service Issue(Problema con il servizio clienti).

Alla luce dei risultati delle tre query precedenti, possiamo affermare che:

- 1) Gli utenti più attivi(dai dati che abbiamo a disposizione) si trovano nel EST degli Stati Uniti
- 2) La Negative_Reason più utilizzata è Customer Service Issue(Problema con il servizio clienti)
- 3) La Negative_Reason più utilizzata per la compagnia United(che è la compagnia che ha ricevuto più tweet negativi, come visto dalla query 3) è Customer Service Issue(Problema con il servizio clienti).